SC3103
Embedded Programming

Lecture 4:

Accessing GPIOs from User Space

Loadable Kernel Modules and Device Drivers

Loke Yuan Ren

yrloke@ntu.edu.sg

S3-B2C-114

User Space vs. Kernel Space

- **User Space**:
  - Runs user applications.
  - Limited access to hardware.
  - Uses system calls to request kernel services.
- **Kernel Space**:
  - Executes privileged operations.
  - Directly manages hardware and resources.
- **System Calls**: The bridge between user and kernel space.

Kernel Responsibilities

- **Process Management**: Schedules and manages tasks.
- **Memory Management**: Allocates and controls RAM.
- **Device Management**: Interfaces with hardware via drivers.
- **Filesystem Management**: Handles data storage and retrieval.
- **Networking**: Manages data transmission and protocols.

Hardware Interfacing in Linux

- Kernel interacts with hardware through **device drivers**.
- Interfaces for user-space access:
  - /dev: Device files for hardware interaction
  - /sys: Kernel information about devices
  - /proc: System information and process details

GPIO

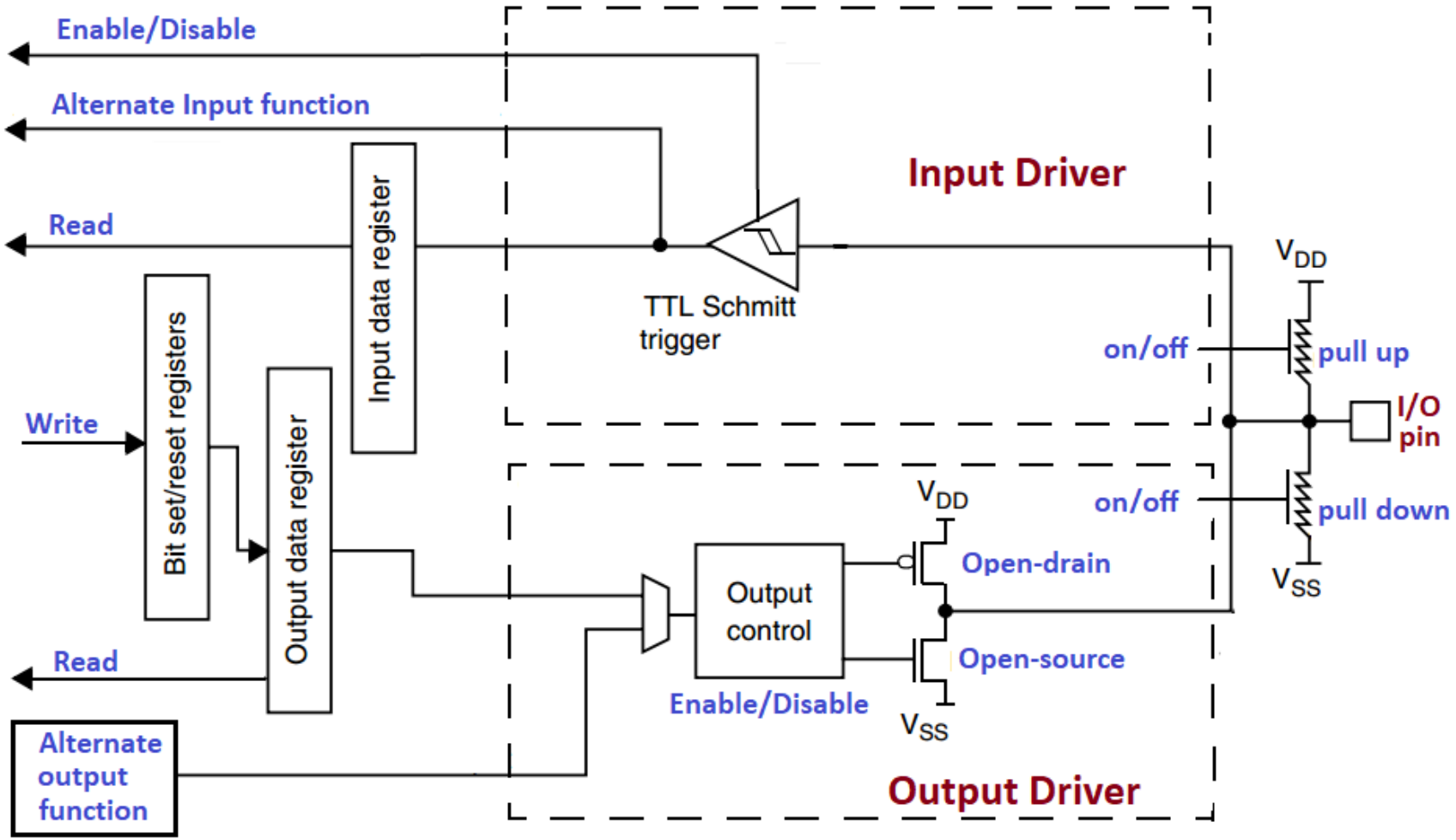
- GPIO: General-Purposes Input/ Output
  - the most commonly used peripheral in embedded systems

- General => consists of generic pins
  - can be configured as input, output or multi-functions
  - can be enabled/disabled
  - can be configured to be source of interrupt

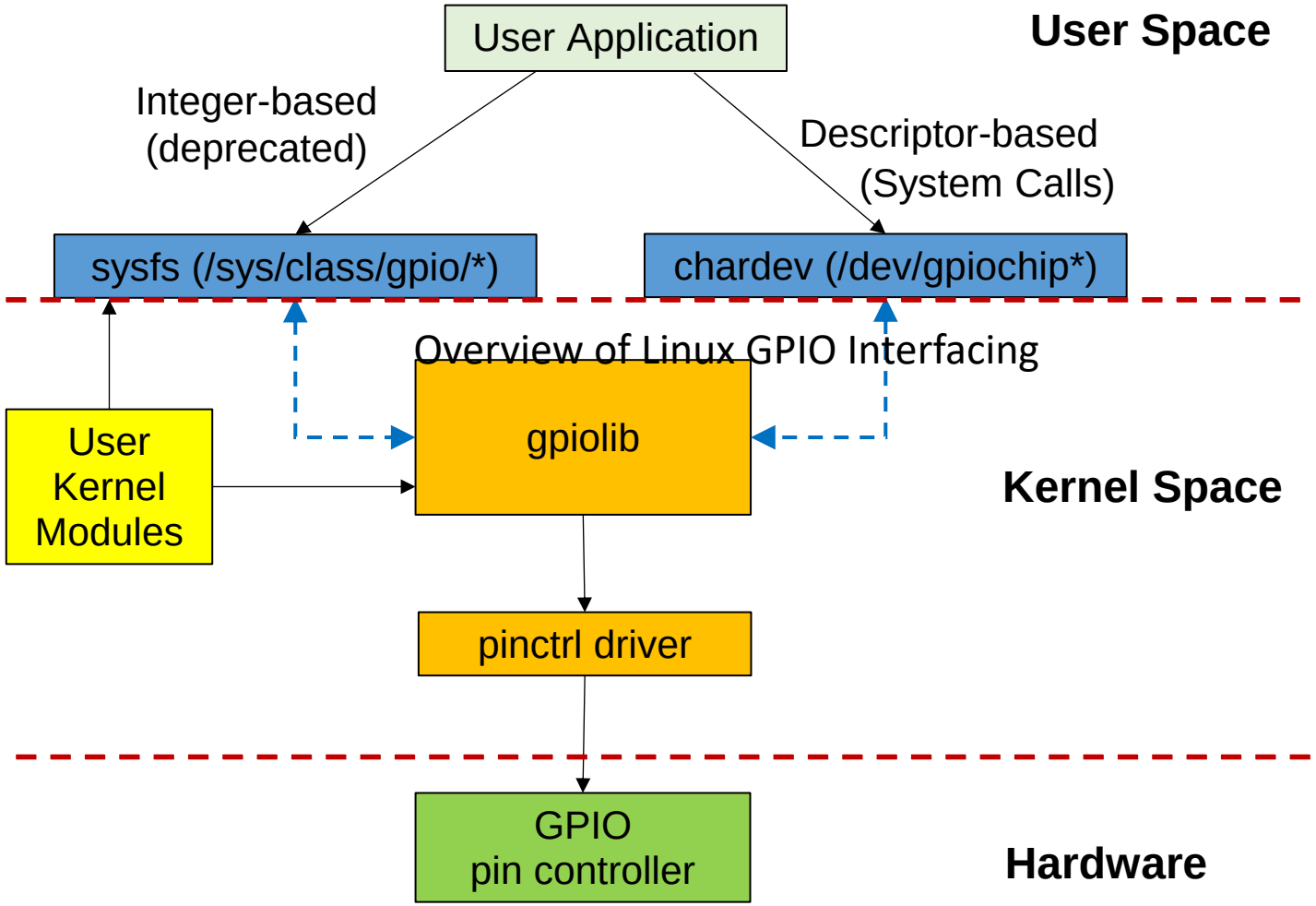
- Usage:
  - LEDS, Buzzers, Relays
  - Switches, Push-buttons, Keypads
  - Multi-functions
  - SPI, I^2C (e.g., sensor devices)
  - PWM (e.g., dimming display, motor drive)

GPIO I/O Pin Structure

An example of typical structure of a GPIO I/O pin

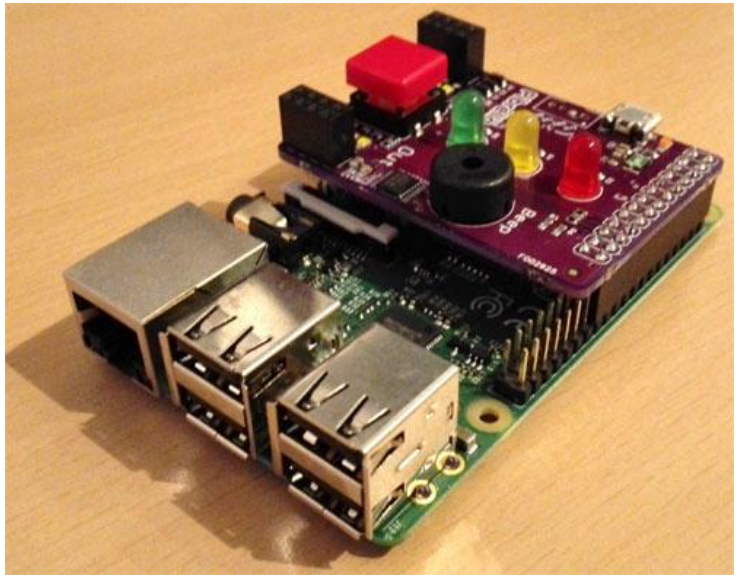


Overview of Linux GPIO Interfacing



User Space Access of GPIOs

- Learn to develop user-space applications for interfacing with GPIO circuits
- Use a descriptor-based character device interface
- Utilize the Pibrella board for GPIO control
- Implement interfacing with the Raspberry Pi



```
pi@raspberrypi:~/ce3103 $ pinout
,--------------------------------.
| oooooooooooooooooooo J8     +====
| 1oooooooooooooooooooo       | USB
|                             +====
|      Pi Model 3B  V1.2         |
|      +----+                 +====
|      |SoC |                 | USB
|  |D| |    |                 +====
|  |S| |    |                    |
|  |I| +----+                    |
|                    |C|     +======
|                    |S|     |   Net
| pwr        |HDMI|  |I||A|  +======
`-| |--------|    |----|V|-------'

Revision           : a22082
SoC                : BCM2837
RAM                : 1024Mb
Storage            : MicroSD
USB ports          : 4 (excluding power)
Ethernet ports     : 1
Wi-fi              : True
Bluetooth          : True
Camera ports (CSI) : 1
Display ports (DSI): 1

J8:
   3V3  (1) (2)  5V
 GPIO2  (3) (4)  5V
 GPIO3  (5) (6)  GND
 GPIO4  (7) (8)  GPIO14
   GND  (9) (10) GPIO15
GPIO17 (11) (12) GPIO18
GPIO27 (13) (14) GND
GPIO22 (15) (16) GPIO23
   3V3 (17) (18) GPIO24
GPIO10 (19) (20) GND
 GPIO9 (21) (22) GPIO25
GPIO11 (23) (24) GPIO8
   GND (25) (26) GPIO7
 GPIO0 (27) (28) GPIO1
 GPIO5 (29) (30) GND
 GPIO6 (31) (32) GPIO12
GPIO13 (33) (34) GND
GPIO19 (35) (36) GPIO16
GPIO26 (37) (38) GPIO20
   GND (39) (40) GPIO21

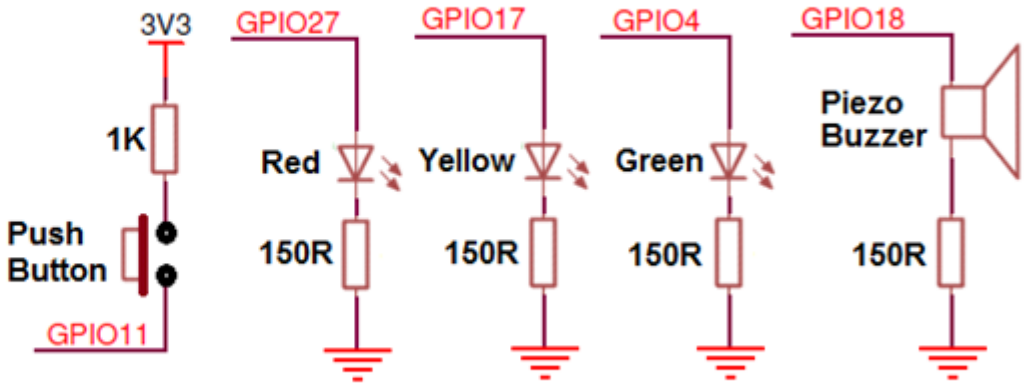
For further information, please refer to https://pinout.xyz/
```

GPIO interfaces for Pibrella

The Pibrella board consists of

- 3 LED displays interfaced through GPIO4, GPIO17 and GPIO27 of RPi board
- 1 Push button input connected through RPi's GPIO11
- 1 Piezo Buzzer driven through RPi's GPIO18





GPIO APIs for User Space Applications

GPIO Access @ User Space

First, check that the GPIO descriptor files are made available by the Kernel at `/dev`

c: character device

```
pi@raspberrypi:~ $ ls -al /dev/gpio*
crw------- 1 root root 254, 0 Feb  6 19:55 /dev/gpiochip0
crw------- 1 root root 254, 1 Feb  6 19:55 /dev/gpiochip1
crw------- 1 root root 254, 2 Feb  6 19:55 /dev/gpiochip2
crw-rw---- 1 root gpio 244, 0 Feb  6 19:55 /dev/gpiomem
```

In the above example (taken from RPi), there are 4 GPIO devices available

- `gpiochip0, gpiochip1, gpiochip2` and `gpiomem`

These are just like normal files, so can be opened by using `open()` system call as follows:

```
int fd0 = open("/dev/gpiochip0",O_RDWR);
```

gpiochip0, gpiochip1, and gpiochip2 in Raspberry Pi

1. gpiochip0 :Main GPIO Controller
   - Controls the main set of Broadcom BCM2711 (or BCM283x) SoC GPIOs.
   - These correspond to the standard Raspberry Pi GPIO header pins.
2. gpiochip1 :GPIO Expander or Auxiliary Controller
   - SPI or I2C-based GPIO expanders
3. gpiochip2 :Additional GPIO Controllers
4. gpiomem :memory-mapped device file for direct access to the GPIO registers

GPIO Access @ User Space (cont)

```
pi@raspberrypi:/usr/include/linux $ cat gpio.h
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
/*
 * <linux/gpio.h> - userspace ABI for the GPIO character devices
 *
 * Copyright (C) 2016 Linus Walleij
 *
 * This program is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 as published by
 * the Free Software Foundation.
 */
#ifndef _GPIO_H_
#define _GPIO_H_

#include <linux/ioctl.h>
#include <linux/types.h>

/**
 * struct gpiochip_info - Information about a certain GPIO chip
 * @name: the Linux kernel name of this GPIO chip
 * @label: a functional name for this GPIO chip, such as a product
 * number, may be NULL
 * @lines: number of GPIO lines on this chip
 */
struct gpiochip_info {
        char name[32];
        char label[32];
        __u32 lines;
};
```

To get the information about the gpio chip

- use the followings which are defined in the gpio header file
  `/usr/include/linux/gpio.h`

```
#include <linux/gpio.h>
#include <sys/ioctl.h> //for ioctl()
#include <fcntl.h>    //for open()
#include <unistd.h>   //for close()
struct gpiochip_info cinfo;
ioctl(fd0, GPIO_GET_CHIPINFO_IOCTL, &cinfo);
fprintf(stdout, "GPIO chip 0: %s, \"%s\", %u GPIO
   lines\n", cinfo.name, cinfo.label, cinfo.lines);
```

```
GPIO chip 0: gpiochip0, "pinctrl-bcm2835", 54 GPIO lines
```

Note that the input/output control system call, `ioctl()` is used to access the device (i.e., the GPIO controller)

- not the usual read() & write().

GPIO Access @ User Space (cont)

The following statements are used to set the I/O pin(s)

- first define the structure to be used by the request handler

```
struct gpiohandle_request req //see gpio.h for detail
req.lines = 3;  // number of lines to request
req.lineoffsets[0] = 4;  //pin 4 - Green LED
req.lineoffsets[1] = 17; //pin 17  - Yellow LED
req.lineoffsets[2] = 27; //pin 27  - Red LED
req.default_values[2] = 1; //default Red LED set to ON
// Configure these pins as output
req.flags = GPIOHANDLE_REQUEST_OUTPUT;
```

- then execute `ioctl()` to get the line handler `req`

```
ioctl(fd0, GPIO_GET_LINEHANDLE_IOCTL, &req)
```

- can then optionally give the lines a name

```
strcpy(req.consumer_label, "RPi LEDs");
```



```
/* Maximum number of requested handles */
#define GPIOHANDLES_MAX 64

/* Linerequest flags */
#define GPIOHANDLE_REQUEST_INPUT        (1UL << 0)
#define GPIOHANDLE_REQUEST_OUTPUT       (1UL << 1)
#define GPIOHANDLE_REQUEST_ACTIVE_LOW   (1UL << 2)
#define GPIOHANDLE_REQUEST_OPEN_DRAIN   (1UL << 3)
#define GPIOHANDLE_REQUEST_OPEN_SOURCE  (1UL << 4)

/**
 * struct gpiohandle_request - Information about a GPIO handle request
 * @lineoffsets: an array desired lines, specified by offset index for the
 * associated GPIO device
 * @flags: desired flags for the desired GPIO lines, such as
 * GPIOHANDLE_REQUEST_OUTPUT, GPIOHANDLE_REQUEST_ACTIVE_LOW etc, OR:ed
 * together. Note that even if multiple lines are requested, the same flags
 * must be applicable to all of them, if you want lines with individual
 * flags set, request them one by one. It is possible to select
 * a batch of input or output lines, but they must all have the same
 * characteristics, i.e. all inputs or all outputs, all active low etc
 * @default_values: if the GPIOHANDLE_REQUEST_OUTPUT is set for a requested
 * line, this specifies the default output value, should be 0 (low) or
 * 1 (high), anything else than 0 or 1 will be interpreted as 1 (high)
 * @consumer_label: a desired consumer label for the selected GPIO line(s)
 * such as "my-bitbanged-relay"
 * @lines: number of lines requested in this request, i.e. the number of
 * valid fields in the above arrays, set to 1 to request a single line
 * @fd: if successful this field will contain a valid anonymous file handle
 * after a GPIO_GET_LINEHANDLE_IOCTL operation, zero or negative value
 * means error
 */
struct gpiohandle_request {
        __u32 lineoffsets[GPIOHANDLES_MAX];
        __u32 flags;
        __u8 default_values[GPIOHANDLES_MAX];
        char consumer_label[32];
        __u32 lines;
        int fd;
};

/**
 * struct gpiohandle_data - Information of values on a GPIO handle
 * @values: when getting the state of lines this contains the current
 * state of a line, when setting the state of lines these should contain
 * the desired target state
 */
struct gpiohandle_data {
        __u8 values[GPIOHANDLES_MAX];
};
```

GPIO Interface @ User Space (cont)

- next define the structure for the lines bit data

```
struct gpiohandle_data data //see gpio.h for detail
data.values[0] = 1; // set initial bit value to high
data.values[1] = 0; // set this to low
```

- then execute ioctl() to set the pins (via `req` & `data`)

```
ioctl(req.fd, GPIOHANDLE_SET_LINE_VALUES_IOCTL, &data)
```

Similar definitions & functions are used to configure pin as input

```
req_pb.lines = 1;
req_pb.lineoffsets[0] = 11;  // push button on pin 11
// Configure this pin as input
req_pb.flags = GPIOHANDLE_REQUEST_INPUT;
ioctl(fd0, GPIO_GET_LINEHANDLE_IOCTL, &req_pb);
// to read the input value
ioctl(req_pb.fd, GPIOHANDLE_GET_LINE_VALUES_IOCTL, &data_pb);
//input bit at data_pb.values[0];
```

GPIO_RPi.c

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <string.h>
#include <unistd.h>
#include <linux/gpio.h>

int main(int argc, char *argv[ ])
{ int fd0 = open("/dev/gpiochip0",O_RDWR);
  struct gpiochip_info cinfo;
  ioctl(fd0,GPIO_GET_CHIPINFO_IOCTL,&cinfo);
  fprintf(stdout, "GPIO chip 0: %s, \"%s\", %u lines\n",
          cinfo.name, cinfo.label, cinfo.lines);
```

```
struct gpiohandle_request req_GY; // Green and Yellow
struct gpiohandle_data data_GY;   // for data bit

req_GY.lines = 2;  // 2 pins in this handler
req_GY.lineoffsets[0] = 4;  //pin 4 - Green LED
req_GY.lineoffsets[1] = 17;  // pin 17 - Yellow LED

// set them to be output
req_GY.flags = GPIOHANDLE_REQUEST_OUTPUT;

// set data values
data_GY.values[0] = 1; // set Green LED to High (ON)
data_GY.values[1] = 0; // set Yellow LED to Low (OFF)

// now get the line handler req_GY
ioctl(fd0, GPIO_GET_LINEHANDLE_IOCTL, &req_GY);
```

```
 // Toggle the two LEDs
  for (int i = 0; i < 5; ++i){ // do it for 5 time
     // output data bits
     ioctl(req_GY.fd, GPIOHANDLE_SET_LINE_VALUES_IOCTL,&data_GY);
     //sleep for 1 second
     usleep(1000000);
     data_GY.values[0] = !data_GY.values[0]; // toggle
     data_GY.values[1] = !data_GY.values[1];
     }
  close(req_GY.fd); // release line
  close(fd0); // close the file
  printf("Done\n");
  exit(EXIT_SUCCESS);
}//main
```

GPIO Event @ User Space

GPIO also supports event request, e.g., from the Push Button

- first define the structure to use as the event request handler

```
struct gpioevent_request req_pb // for push button
struct gpioevent_data event;
req_pb.lineoffset = 11;  //pin 11 push button
req_pb.handleflags = GPIOHANDLE_REQUEST_INPUT;
// using Rising Edge to trigger event
req_pb.eventflags = GPIOEVENT_REQUEST_RISING_EDGE;
strcpy(req_pb.consumer_label, "Push Button Event");
```

- then execute `ioctl()` to get the event handler `req_pb`

```
ioctl(fd0, GPIO_GET_LINEEVENT_IOCTL, &req_pb);
```

- and wait for event to occur (blocking)

```
read(req_pb.fd, &event, sizeof(event));
```

- to print timestamp of the occurrence of the event

```
printf("Event occur @ %"PRIu64"\n", event.timestamp);
```

```
/* Eventrequest flags */
#define GPIOEVENT_REQUEST_RISING_EDGE   (1UL << 0)
#define GPIOEVENT_REQUEST_FALLING_EDGE  (1UL << 1)
#define GPIOEVENT_REQUEST_BOTH_EDGES    ((1UL << 0) | (1UL << 1))

/**
 * struct gpioevent_request - Information about a GPIO event request
 * @lineoffset: the desired line to subscribe to events from, specified by
 * offset index for the associated GPIO device
 * @handleflags: desired handle flags for the desired GPIO line, such as
 * GPIOHANDLE_REQUEST_ACTIVE_LOW or GPIOHANDLE_REQUEST_OPEN_DRAIN
 * @eventflags: desired flags for the desired GPIO event line, such as
 * GPIOEVENT_REQUEST_RISING_EDGE or GPIOEVENT_REQUEST_FALLING_EDGE
 * @consumer_label: a desired consumer label for the selected GPIO line(s)
 * such as "my-listener"
 * @fd: if successful this field will contain a valid anonymous file handle
 * after a GPIO_GET_LINEEVENT_IOCTL operation, zero or negative value
 * means error
 */
struct gpioevent_request {
        __u32 lineoffset;
        __u32 handleflags;
        __u32 eventflags;
        char consumer_label[32];
        int fd;
};

/**
 * GPIO event types
 */
#define GPIOEVENT_EVENT_RISING_EDGE 0x01
#define GPIOEVENT_EVENT_FALLING_EDGE 0x02

/**
 * struct gpioevent_data - The actual event being pushed to userspace
 * @timestamp: best estimate of time of event occurrence, in nanoseconds
 * @id: event identifier
 */
struct gpioevent_data {
        __u64 timestamp;
        __u32 id;
};

#define GPIO_GET_CHIPINFO_IOCTL _IOR(0xB4, 0x01, struct gpiochip_info)
#define GPIO_GET_LINEINFO_IOCTL _IOWR(0xB4, 0x02, struct gpioline_info)
#define GPIO_GET_LINEHANDLE_IOCTL _IOWR(0xB4, 0x03, struct gpiohandle_request)
#define GPIO_GET_LINEEVENT_IOCTL _IOWR(0xB4, 0x04, struct gpioevent_request)
```

Libgpiod

Newer Linux release supports the libgpiod C library and tools

- abstract the gpio functions to simpler C APIs (in gpiod.h)
- also provide command-line tools: `gpiodetect, gpioinfo, gpioset, gpioget, gpiofind, gpiomon`

Examples:

```
$gpiodetect
gpiochip0 [pinctrl-bcm2835] (54 lines)
              :
$gpioinfo gpiochip0
gpiochip0 - 54 lines:
        line 0: "SDA0" unused input active-high
        line 1: "SCL0" unused input active-high
        line 2: "SDA1" unused input active-high
        line 3: "SCL1" unused input active-high
        line 4: "GPIO_GCLK" unused input active-high
               :
```

Summary

GPIOs can be accessed at the user space through the ‘new’ descriptor based char device file

- one device file per gpiochip
  - at `/dev/gpio*`
- access the device using the `open(), ioctl()` and `close()`
- can operate multiple lines simultaneously on a device file
  - setting and reading of I/O bits
- also support event request

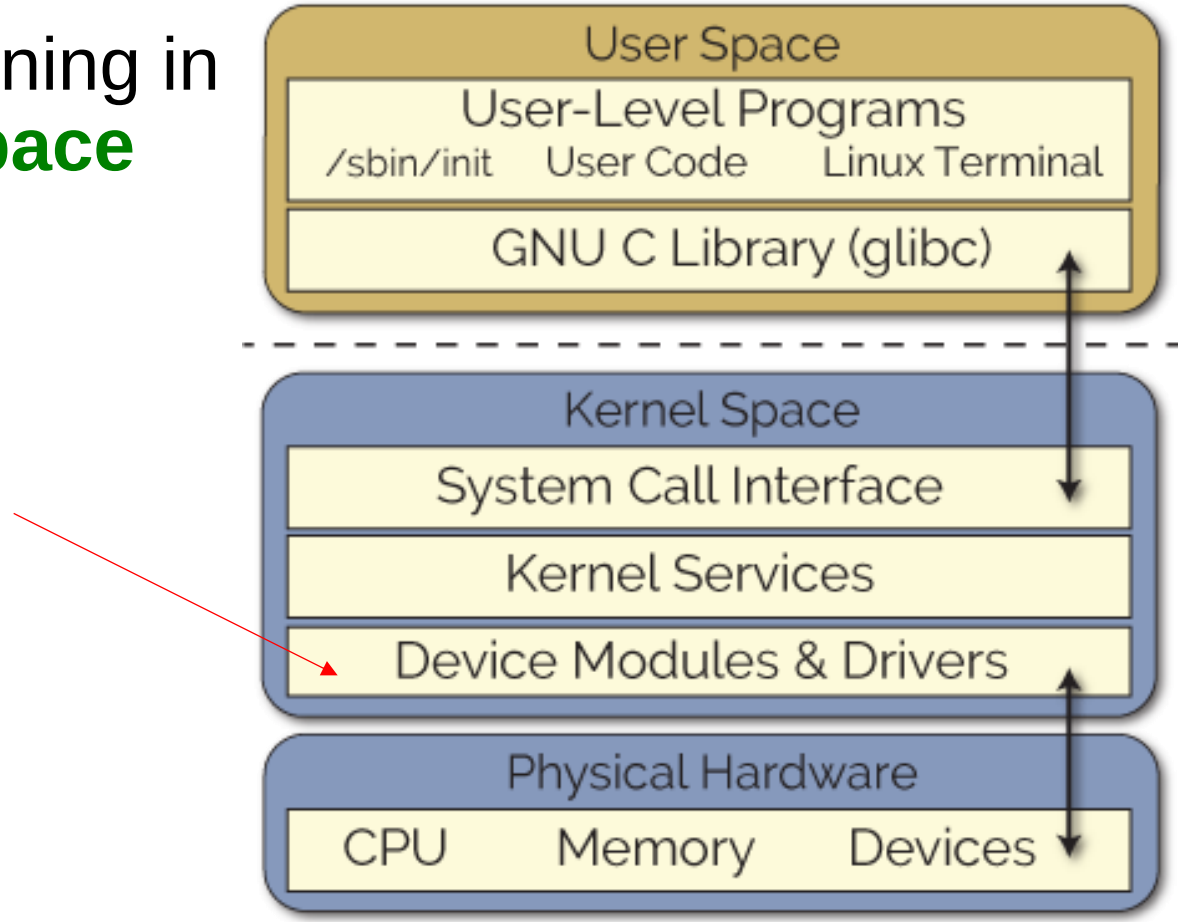
Additional Info: vcgencmd commands

- Provides access to various hardware metrics and settings
  - Temperature: vcgencmd measure_temp
  - Voltage: vcgencmd measure_volts
  - Clock: vcgencmd measure_clock arm
- To check for undervoltage issues: vcgencmd get_throttled
  - 0x0: normal.
  - 0x50000: undervoltage has been detected.
  - 0x20000: throttling due to high temperature.
- dmesg | grep -i voltage
- Using **vcgencmd** in scripts for automation.
- Logging and analyzing Raspberry Pi performance over time.

Loadable Kernel Modules

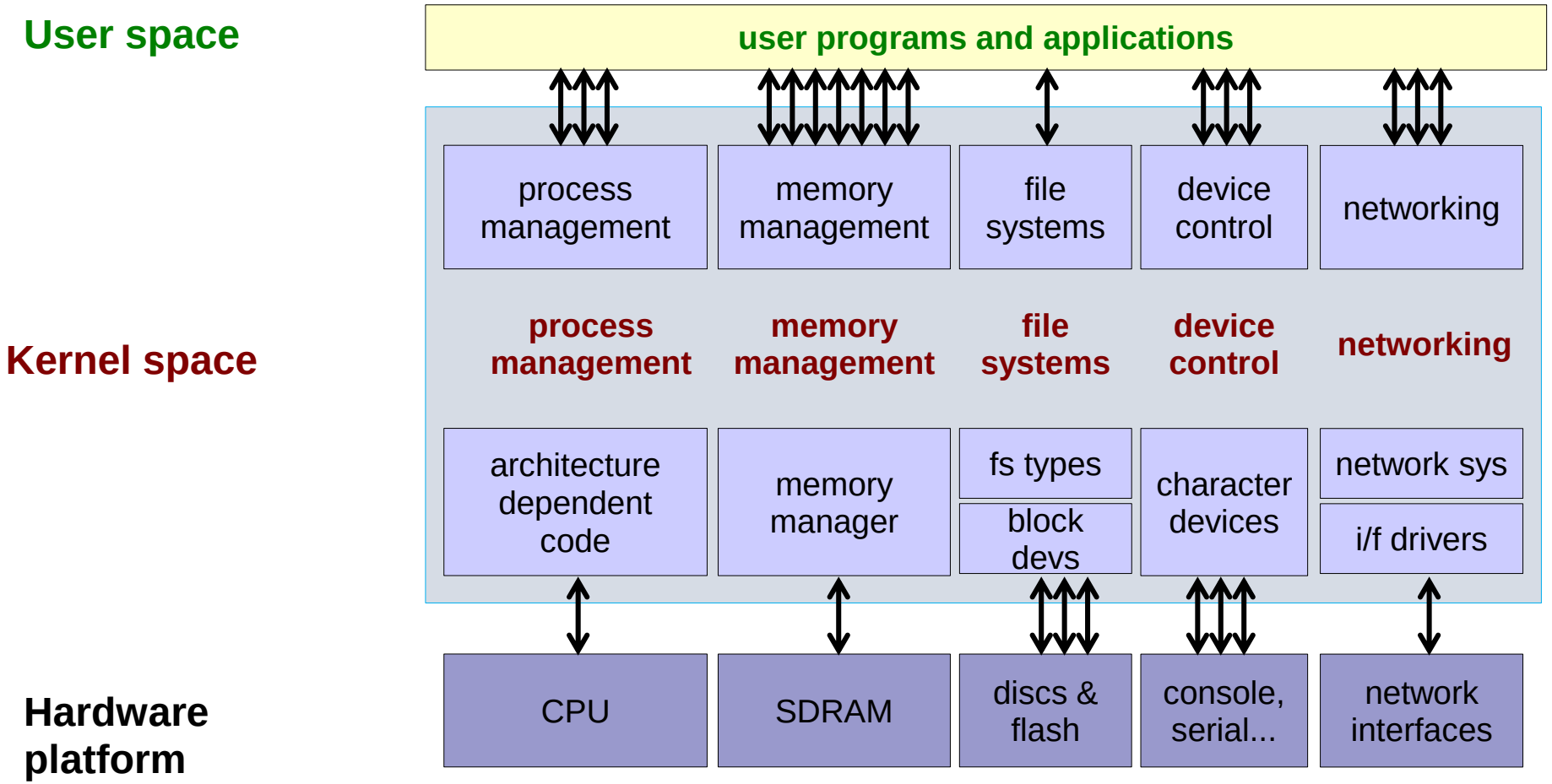
Kernel Space and User Space

In Linux system, code can be running in
- either **kernel space** or **user space**



Reference: Derek Molloy

In a bit more detail



Kernel Space and User Space (cont)

User space code is what you run from the command line
- either as a user or as root.
- typically are programs that you write in 'C' or other languages, compile and execute

Kernel space code is used for purposes such as
- low level OS routines and scheduling (Kernel)
- basic I/O access to the hardware (via Kernel modules)

Kernel memory and code space is **protected** from the user
- user space program can only access the kernel space through executing system calls (e.g. open())

In fact, some program functions (e.g. fopen()) provided by standard library (e.g. glibc)
- actually execute the appropriate system calls behind the ‘scene’. (e.g. fopen() is a ‘wrapper’ for syscall open().)

Loadable Kernel Module

Loadable Kernel module (LKM) is a piece of code that can be loaded and unloaded into the kernel space upon demand at run time

- hence extend the functionality of the kernel without the need to reboot the system

One type of LKMs that we will study here is the device driver

- which allows the kernel to access hardware connected to the system – more on this later

Without modules, we would have to build monolithic kernels and add new functionality directly into the kernel image

- lead to larger kernels as new functionalities are added.
- require us to rebuild and reboot the kernel every time we want to add new functionality – time consuming.

Superuser for Modules Access

At this point we should note that most users in a Linux system do not have sufficient privilege

- to access module and perform hardware I/O access
- only root (superuser) privilege can do so.

Many of the examples in this topic need root privilege,
and there are two ways to achieve this:

1) Run command with root privilege (if the system is set up for this):
   `sudo <command>` (when prompted, enter your user password)
2) Become root in the current shell:
   `su` (when prompted, enter root password)

   For ubuntu based system which usually has no root password:
   `sudo su` (when prompted, enter your user password)

Note: To exit from root – type **`exit`**
To check current privilege level - type **`whoami`**

Interfacing to the Real World

Linux as an Embedded OS almost always needs to interface with the outside world (i.e. with I/O functionality).

According to our discussion so far

- this is done through kernel space.

So how can we write a program that can accesses I/O data

- if the I/O is in kernel space, and the program we run are in user space?

1. Do we write code for kernel space, user space or both?

2. How do we write code for these two spaces?

3. How do we communicate between spaces?

Interfacing to the Real World (cont)

1. Do we write code for kernel space, user space or both?

Actually as long as we have root access, we can probably do *either*.

If we have to handle some real world I/O we could:

i. have all the code in kernel space
   - i.e. by adding code to the kernel,
   - but difficult to write and debug

ii. write a kernel space driver that sends/receives I/O data to/from a user space handler program
   - which is what we will learn here

iii. write a user space program and issue system calls to access the special 'file' made available by the OS

Interfacing to the Real World (cont)

2. So how do we write code for these two spaces?

User space code is simple

- just write a program, compile it and run it at the command line (or from the rcS script*).

Kernel space code is either

- written as a permanent part of the kernel (a static driver)

  or

- a *runtime-linkable* block of object code: a dynamic (device) driver, or loadable kernel module, LKM
  (i.e. a *DLL* in Windows terminology).

3. How do we communicate between spaces?

We will answer that later.

But let's start with creating a simple kernel module.

Basic Kernel Module Structure

A minimal kernel module program structure would contain the following:

- Kernel specific header file: <linux/xxxx.h>
  - because we don't have access to the normal user space's C library
- an initialization function
  - called when a module is loaded (e.g. using the insmod command)
- a cleanup function
  - called when a module is unloaded (e.g. using the rmmod command)
- **`module_init()`**
  - macro used to declare the initialization function

Basic Kernel Module Structure (cont)

- **`module_exit()`**
  - macro used to declare the cleanup function

- Macros for metadata information that provide detail about the module (access using modinfo):

```
MODULE_LICENSE()
MODULE_AUTHOR()
MODULE_DESCRIPTION()
MODULE_VERSION()
```

Basic Kernel Module

Some notes about kernel module:

Kernel modules are written in C

- but they are not programs
  E.g., there is no `main()` function in a module

Kernel modules are typically event driven

- rather than executing sequentially.

Example: Hello World Kernel Module

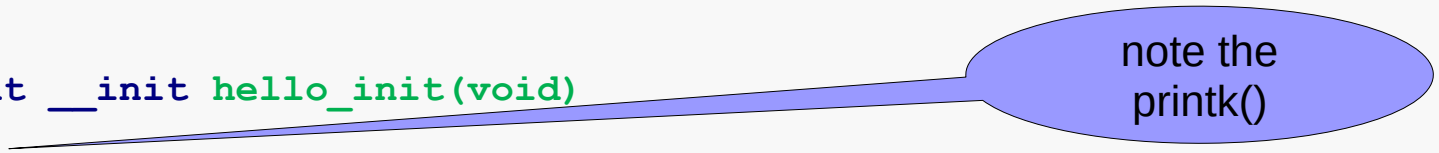
```
/* modhelloV1.c */
#include <linux/init.h>   /* needed by the macrso module_init and exit */
#include <linux/module.h> /* needed by all modules */
#include <linux/kernel.h> /* needed by KERN_ definition*/


static int __init hello_init(void)
{
  printk(KERN_ALERT "Hello from kernel world – V1\n");
  return 0; /*0 for success, negative for failure */
}

static void __exit hello_exit(void)
{
  printk(KERN_ALERT "Goodbye from kernel world - V1\n");
}

module_init(hello_init); /* macro to execute module's initialize routine */
module_exit(hello_exit); /* macro to execute module's exit routine */

MODULE_LICENSE("GPL");
MODULE_AUTHOR("SC3103");
MODULE_DESCRIPTION("Simple HW module");
MODULE_VERSION("V1");
```



The detail - ‘printk()’

Let’s look at the code:

```
printk(KERN_ALERT "Hello from kernel world\n");
```

Inside the kernel you can't use shared library functions!

- these are all in user-space.
- **printk()** is the **printf()** equivalent for use in kernel space.

You can specify a priority level: e.g. **KERN_ALERT** which is <1> , and depending on this and other setting, the text get printed to

- the console

or

- the system and/or message log: syslog or kern.log in var/log/ (or /var/log/messages if enabled)

The dmesg command can also be used to view these messages.

(This assumes syslogd and klogd are running - these are background processes that handle system and kernel logging.)

Type “**dmesg | tail**” to see the message printed by the module when it loaded.
Any printk message will be sent to the kernel log file: type **“tail /var/log/kern.log”**
Other module utilities: “**lsmod | grep “mod**”, “**modinfo modhello.ko**”

Printk() and its alias functions

Name	Alias function
KERN_EMERG	pr_emerg()
KERN_ALERT	pr_alert()
KERN_CRIT	pr_crit()
KERN_ERR	pr_err()
KERN_WARNING	pr_warn()
KERN_NOTICE	pr_notice()
KERN_INFO	pr_info()
KERN_DEBUG	pr_debug() and pr_devel() if DEBUG is defined
KERN_DEFAULT	
KERN_CONT	pr_cont()

Macros ‘__init’ & ‘__exit’

__init macro

- Marks a function as *initialization-only*. For **built-in drivers** (compiled into the kernel): The function is used only during system startup.
- Its memory is **freed after initialization**, reducing kernel memory usage.
- For **loadable kernel modules (LKMs)**: The function is executed when the module is loaded.

__exit macro

- Marks a function as cleanup/exit code.
- For LKMs:
  - The function is executed when the module is unloaded (e.g., via rmmod).
- For built-in drivers:
  - The function is not needed and is omitted to save space.

MODULE_LICENSE("GPL")

- Declares the module’s license. Accessible via tools like modinfo. Important because: Indicates the module complies with **GPL (free software)**.
- Allows access to certain **GPL-only kernel symbols**.
- Helps the kernel determine whether the module is “tainted”.

Building a Module

Two ways to build a module:

i. inside a kernel's own source tree
   - integrated into the kernel configuration and compiling process
   - can build the module statically in kernel
   - involved modification of the kernel configuration menu (complicated)

ii. 'out-of-tree'
   - module code remains in its own directory
   - but built by using the kernel module's build utilities
   - generally more convenient – we will use this approach.

A successful compiled module output
   a binary module with **`.ko`** suffix

(But module must be compiled against the kernel on which it is to execute on – more on this later)

Recap: Makefile

As you have learnt earlier, you compile your C source code at the console/terminal by using the command:

```
gcc  -o  myprog  myprog.c
```

If you have more than one source files

- you can build all of them together:

```
gcc  -o  myprog  myprog_partA.c  myprog_partB.c
```

- or use the **`Makefile`**
  - ➢ a file that contains the information on how the file should be compiled (and other features that is convenient for developing large programs)

Compiling using the **`Makefile`** is then done simply by issuing the command:

```
make
```

Out-of-tree Compiling for LKM

Steps to build the module outside of the kernel source tree.

i) A special `Makefile` (call *kbuild Makefile*) is required to build the kernel module. Example:

```
obj-m := modhelloV1.o
 all:
        make -C /lib/modules/$(shell uname –r)/build M=$(PWD) modules
clean:
        make -C /lib/modules/$(shell uname –r)/build M=$(PWD) clean
```

- The 1st line defines the loadable module to be built (known as goal definition)
  obj-m := modhelloV1.o
- The 3rd line defines the rule to build the module
  - if we have the kernel headers installed, you also can
    make -C /usr/src/linux-headers-`uname -r` M=`pwd` modules
  - if we have kernel source installed, you also can
    make -C /usr/src/kernel-source-`uname -r` M=`pwd` modules

ii) -C $(KERNEL_SOURCE) tells make to first change into the kernel source directory

iii) Type *make* to make/build the module

Demo

We will learn later how to compile a kernel module program to produce its kernel module file with extension **.ko**

- E.g. for the previous program
  modhelloV1.c ⇒ **modhelloV1.ko**

➢ To 'execute' this module, use the command:
  `sudo` **insmod ./modhelloV1.ko**

➢ To check active modules: **lsmod | egrep "mod"**

➢ To see the message(s) output by module: **dmesg | tail**
  or **tail -f /var/log/kern.log**

➢ To get the information about the module:
  modinfo modhelloV1.ko

➢ To remove the module: `sudo` **rmmod modhelloV1**

Module Handling Utilities

`insmod ./mymod.ko`

- to insert the new module (you must give a full path, otherwise it looks in /lib/modules to find the module)

`lsmod` (or cat /proc/modules)

- to list the set of modules currently loaded, plus give size, usage count and any running modules that depend on it.

`rmmod mymod`

- to remove the named module. Note the name is that shown in the lsmod listing, not the filename!

`modprobe`

Loads a module and its dependencies.

(Not necessary for stand-alone module development.) Dependency information is stored in /etc/modules.conf by the *depmod* command.

Module Handling Issues

Important information:
1. Only the root user can load/unload modules.

2. Module and kernel version numbers are VERY important.

3. If your module crashes it might take down the kernel.

4. These days you need to decide if your modules are GPL'd or not, otherwise they will *taint* the kernel
   - loading a proprietary or non-GPL-compatible LKM will set a 'taint' flag in the running kernel
   - meaning that any problems/bugs experienced will be less likely to be investigated by the maintainers.

Module License and Symbols

Based on the metadata indicated in its **`MODULE_LICENSE()`**

- a module could be declared as GPL-licensed module

A function (e.g. **`func1()`**) declared in kernel (and modules) can be made visible (i.e. exported) to other modules using the macros:

- `EXPORT_SYMBOL(func1)`

But can restrict a function (e.g. **`func2()`**) to be visible only to GPL-licensed module, by using the macro:

- `EXPORT_SYMBOL_GPL(func2)`

Hence can restrict the functions that a non GPL-licensed module can access.

Exporting Module

```
#include <linux/module.h>
#include <linux/kernel.h>
void exported_function(void) {
    pr_info("Hello from exported_function\n");
}

EXPORT_SYMBOL(exported_function);

static int __init exported_init(void) {
    pr_info("Exported module loaded\n");
    return 0;
}
```

```
static void __exit exported_exit(void) {
    pr_info("Exported module unloaded\n");
}

module_init(exported_init);
module_exit(exported_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("Module that exports a function.");
```

Importing Module

```
#include <linux/module.h>
#include <linux/kernel.h>

extern void exported_function(void);

static int __init imported_init(void) {
    pr_info("Imported module loaded\n");
    exported_function();  // Call the function
    return 0;
}

static void __exit imported_exit(void) {
    pr_info("Imported module unloaded\n");
}

module_init(imported_init);
module_exit(imported_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("Module that imports a function.");
```

Export/Import Modules

- EXPORT_SYMBOL(exported_function); makes the function available.
- The importing module declares it as extern and calls it.
- You must load exported.ko before imported.ko.
- You also must remove imported before exported

Passing Load-time Parameters to Module

Module can be more useful if it accepts parameters during the loading of the module

- E.g. to modify its behaviour, such as to put it in debug mode, verbose reporting etc.

Example: we can declare a variable **debug_enable** for a module as follows:

```
static int debug_enable = 0;
module_param(debug_enable, int, 0);
MODULE_PARM_DESC(debug_enable, "Enable debug
                 mode.");
```

Parameter can then be passed during insmod by adding it after the module name:

```
sudo insmod ./modhelloV2.ko debug_enable=1
```

Passing Parameter to Module

```
/* modhelloV2.c */
#include <linux/init.h>   /* needed by the macros module_init and exit */
#include <linux/module.h> /* needed by all modules */
#include <linux/kernel.h> /* needed by KERN_ definition*/

/* passing parameter to module */
static int debug_enable = 0;              /* default is disabled/off */
module_param(debug_enable, int, 0);       /* macro defined in module.h)*/
MODULE_PARM_DESC(debug_enable, "Enable debug mode");

static int __init hello_init(void)
{ printk(KERN_ALERT "Hello from kernel world - debug mode is %s\n",
  debug_enable ? "enabled" : "disabled");
  return 0;
}

static void __exit hello_cleanup(void)
{ printk(KERN_ALERT "Goodbye from kernel world - v2\n");
}

module_init(hello_init);
module_exit(hello_cleanup);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("CE3103");
MODULE_DESCRIPTION("Simple module - v2");
```

Interfacing Between the Two Spaces

3. But how do we continue to communicate between spaces once the module is loaded?

In Linux, user space code can interface to kernel space drivers in three very well-defined ways:

i. Through filesystem nodes

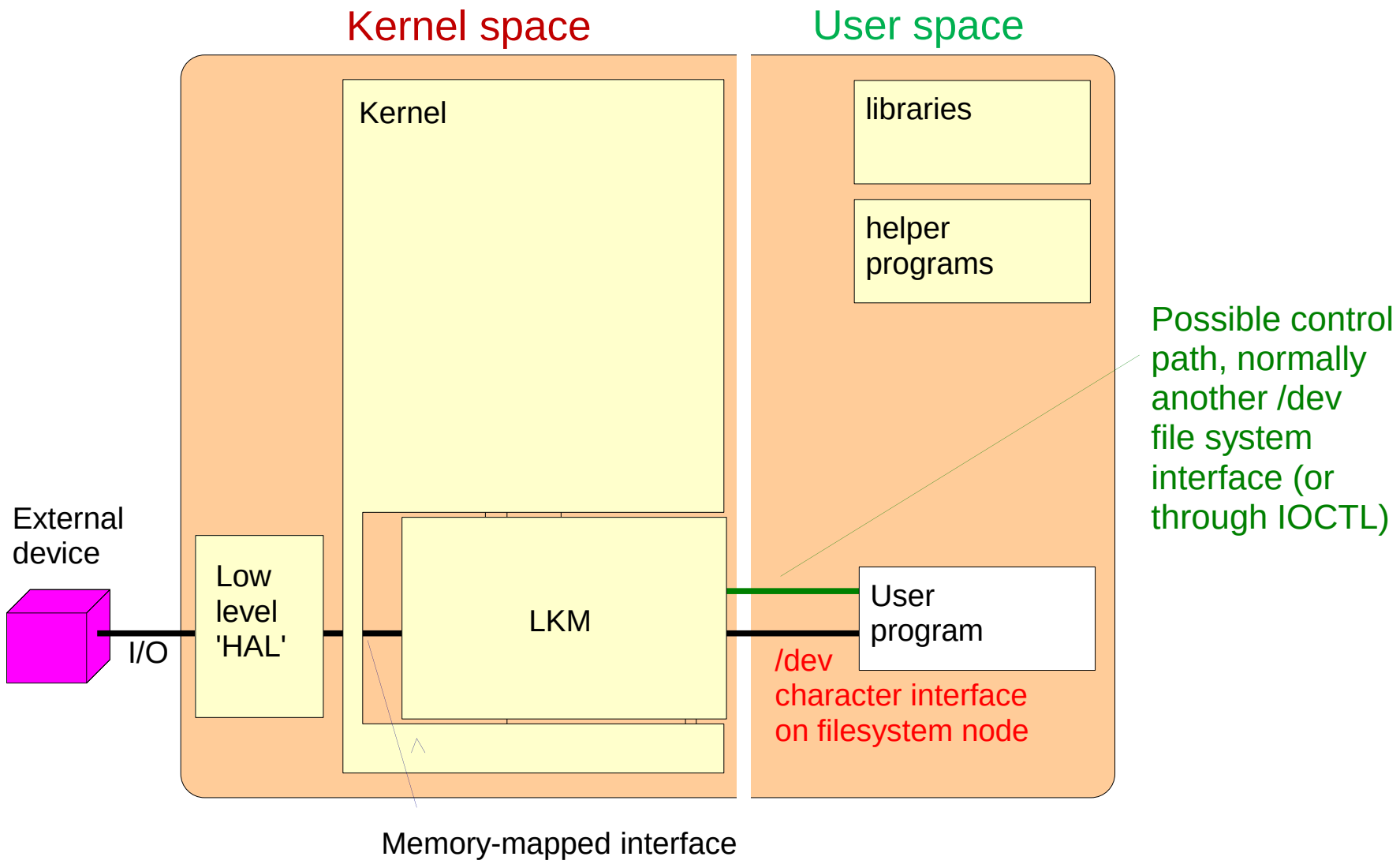
ii. Through IOCTLs (I/O control)

iii. Socket/packet interfaces

Communication Between the Two Spaces

- **Communication can be done through ‘nodes’ in the filesystem**
- A module, when loaded, appears in the filesystem
- i.e. just like a file, except with 0 length.
- They can then be opened, copied, renamed, deleted, read from and written to just like any other file
- remember *“In Unix/Linux, everything is a file” (*from user space perspective)
- **Examples:** In the device directory **`/dev`**, you will see
  - **`/dev/ttyS0`**
  - **`/dev/mem0`**
  - **`/dev/null`**
  - **`/dev/random`**
  - **`/dev/audio`**
  - **`/dev/dvd`**
  - **`/dev/zero`**
  - **`/dev/rtc`**
- and a lot more. (e.g. gpiochip in RPi)

Interfacing to the Real World



Device Node

A **Device node** is a special file that represents a device.

Filesystem nodes have a ***type***, a ***major*** number and a ***minor*** number.

E.g. Do `ls -l /dev/ttyS0` to find this information:

crw-rw---- 1 root dialout 4, 64 date time /dev/ttyS0

They can be automatically created by a LKM when it loads, or can be created by the root user of a system using the 'make node' command:

```
mknod /dev/ttyS1 c 4 65
```

Meaning:
*Its type is a **character** device.*
*Its **Major number** is 4*
*Its **Minor number** is 64 for ttyS0 (and 65 for ttyS1)*

Device Types

Aspect	Character Device	Block Device	Network Device
Data Access	Byte-by-byte, sequential	Block-based, random access	Packet-based, abstracted by OS
Examples	Serial ports, terminals	Hard drives, SSDs	Ethernet, Wi-Fi
Buffered	No	Yes	Yes
I/O Type	Unbuffered	Buffered	Packet-oriented
File System Integration	No	Yes	No
Common Commands	cat, echo, ioctl	dd, mount, mkfs	ip, ifconfig, ping, socket

Most commonly used interface

- character device, which is used to transfer data
- block device, which is used to store or hold data

Registering a Module

We will stick with **character devices** here.

When an LKM is loaded, the first thing it does is to register with the kernel to “own” a particular device major number.

The kernel module function to register a module (which is declared in <*linux/fs.h*>) :

```
register_chrdev(MAJOR_NUM, DEVICE_NAME, &fops)
```

Side note: A special case is when MAJOR_NUM = 0, then the kernel can dynamically allocate a MAJOR number using

```
alloc_chrdev_region().
```

Very useful for some architectures which are running out of MAJOR numbers!

To remove the driver from the system:

```
unregister_chrdev(MAJOR_NUM, DEVICE_NAME)
```

Major and Minor Numbers

Whenever there is a command to access a device by a *user space* program, i.e. to open, read from or write to the device file

- the kernel looks at the MAJOR number
- and then looks in its internal registry to find the module that has registered to handle that MAJOR number.

Hence the major number is used to identify the module (driver) associated with the device.

Kernel then passes control to the corresponding module, plus a pointer to a data structure with lots of info such as:

- what is its MINOR number?
- number of chars requested/sent
- pointer to memory holding/waiting for the chars

Module I/O Access

To handle the normal user space file request operations (via system calls) such as

- open device, close device, read device, write device

and other functions to be called on like

- load module, unload module, on receipt of ioctl etc...

LKMs must implement a corresponding set of functions, known as **driver methods**, to support the operations

- E.g. **`open(),release(),read(),write()`**

The supported operations by a module

- is defined in the file_operations data structure

A module only needs to implement the relevant device methods for its required operations.

file_operations Data Structure

This file_operations data structure is defined in linux/fs.h,

- holds pointers to functions (known as methods) within the driver that allow you to perform various operations on the device.

```
struct file_operations
{   struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    ssize_t(*aio_read) (struct kiocb *, char __user *, size_t, loff_t);
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    ssize_t(*aio_write) (struct kiocb *, const char __user *, size_t,
    int (*readdir) (struct file *, void *, filldir_t);
    unsigned int (*poll) (struct file *, struct poll_table_struct *);
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, struct dentry *, int datasync);
    int (*aio_fsync) (struct kiocb *, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    ssize_t (*readv) (struct file *, const struct iovec *, unsigned long, loff_t *);
    ssize_t (*writev) (struct file *, const struct iovec *, unsigned long, loff_t *);
    ssize_t(*sendfile) (struct file *, loff_t *, size_t, read_actor_t, void __user *);
    ssize_t(*sendpage) (struct file *, struct page *, int, size_t, loff_t *, int);
    unsigned long (*get_unmapped_area) (struct file *, unsigned long, unsigned long,
    unsigned long, unsigned long);
};
```

file_operations Data Structure

We will look more closely at the **highlighted** driver methods.

- methods that are not implemented in the program will point to NULL and become inaccessible.

```
struct file_operations
{
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    ssize_t(*aio_read) (struct kiocb *, char __user *, size_t, loff_t);
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    ssize_t(*aio_write) (struct kiocb *, const char __user *, size_t,
                :
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, struct dentry *, int datasync);
                :
};
```

Basic Driver Methods

The *open* method is called when a file is opened

- provided for a driver to do any initialization in preparation for later operations.

The *release* method is the reverse of *open*

- which is to deallocate any data structure performed by *open* and then shut down the device.

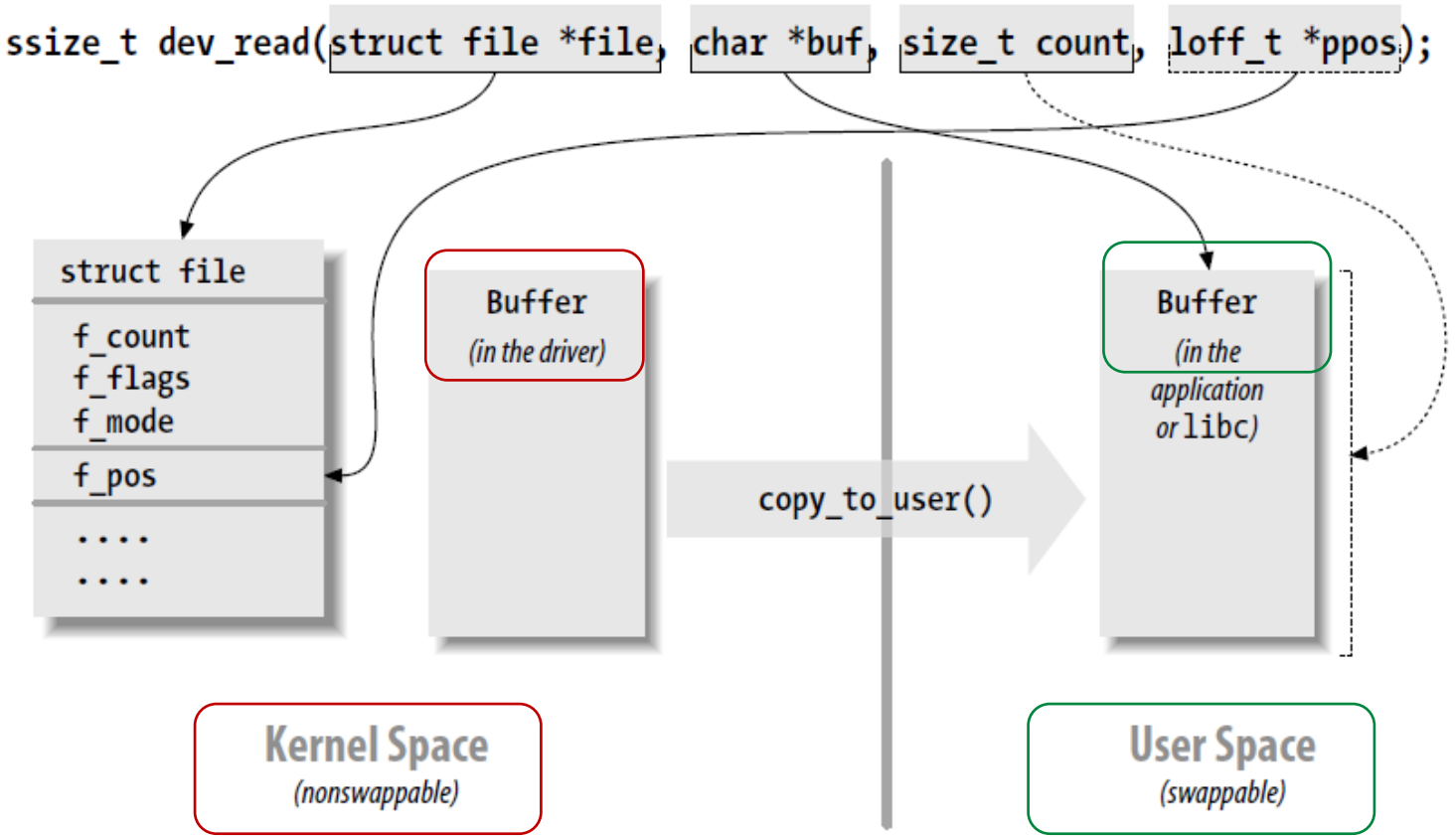
The *write* method copies data from user space application code to kernel space device

The *read* method copies data from the kernel space device to user space application code.

For methods that are not implemented in a driver

- the corresponding entries in the file_operations structure should be set to NULL (i.e. not declared)

Example: read Method's arguments



Arguments for the Read method
(source: Linux Device Drivers – Rubini)

Writing a module with I/O access

Now we will add file I/O functionality to the earlier module.

First of all we will register a character driver called /dev/ce3103 and give it a static major number 101
(we should look in /usr/include/linux/major.h to see if 101 is spare. *But it would be better if we ask the kernel to give us a dynamic number*).

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>

static struct file_operations fops = {};

static int __init ce3103_init(void)
{
   printk(KERN_ALERT "Loading modIO\n");
   register_chrdev(101, "ce3103", &fops);
   return(0);
}
```

Writing a module with I/O access

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>

static struct file_operations fops = {};

static int __init ce3103_init(void)
{  printk(KERN_ALERT "Loading modIO\n");
   register_chrdev(101, "ce3103", &fops);
   return(0);
}

Static void __exit ce3103_cleanup(void)
{  unregister_chrdev(101, "ce3103");
   printk(KERN_ALERT "Unloading modIO\n");
}

module_init(ce3103_init);
module_exit(ce3103_cleanup);
```

Writing a module with I/O access

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h> /* for put_use and get_user */

static int device_open(struct inode *, struct file *);
static int device_release(struct inode *, struct file *);
static ssize_t device_read(struct file *, char *, size_t, loff_t *);
static ssize_t device_write(struct file *, const char *, size_t, loff_t *);

static struct file_operations fops = { .read = device_read,
                                       .write = device_write,
                                       .open = device_open,
                                       .release = device_release
                                     };

char msg_buf[20], ctr = 0; // buffer to store message from user space
char msg1[7]="SC3103"; // message to be senT to the user space
int errorno, msg1len =6;

static int __init ce3103_init(void)
{   printk(KERM_ALERT "Loading modIO\n");
    register_chrdev(101, "sc3103", &fops);
    return(0);
}
```

Writing a module with I/O access

```
static void __exit sc3103_cleanup(void)
{    unregister_chrdev(101, “sc3103”);
     printk(KERN_ALERT “Unloading modIO\n”)
}

static int device_open(struct inode *inode, struct file *file)
{  printk(KERN_ALERT “Someone opened modIO\n”);
   return 0; }

static int device_release(struct inode *inode, struct file *file)
{  printk(KERN_ALERT “Someone closed modIO\n”);
   return 0; }

static ssize_t device_read(struct file *filp, char *buf, size_t len,
                           loff_t *off)
{   printk(KERN_ALERT “reading the device 101\n”);
   if (ctr==0)
      return 0;              /* nothing to return to userland*/
    errorno = copy_to_user(buf, msg1, msg1len);  /* write to user space */
    ctr=0;
    return msg1len;           /* return size of message */
}
```

Writing a module with I/O access

```
static ssize_t device_write(struct file *filp, const char *buf, size_t len,    loff_t *off)
{   printk(KERN_ALERT “writing to device 101\n”);
    if (ctr>0)
       return 0;     /* no space to accept another char */
    errorno = copy_from_user(msg_buf, buf, 12); /* retrieve value from user space
    ctr=1;
    return 1;     /* 1 message read from buffer*/
}
```

Note that the buffers, buf are in user memory, not kernel memory

- so direct pointer assignment won't work.

We therefore have to use functions like *copy_to_user()* and *copy_from_user()* to copy between the memory spaces.

Accessing User Space from Kernel Space

There are a number of functions that can be used by kernel code to ‘move’ (copy) data to and from user space (declared in <linux/uaccess.h>)

- *copy_from_user (void *to, const void *from, unsigned long count);*
- *copy_to_user (void *to, const void *from, unsigned long count);*

For small data size transfer

- use *put_user(datum, ptr)* and *get_user(datum, ptr)*
- relatively fast and should be called (instead of *copy_to_user*) whenever single values are being transferred.

Creating A Device Node

The module is loaded, but just sits there doing nothing

- we can 'talk to it' through a filesystem node

But first we must create that node, with major number 101:

```
sudo mknod /dev/sc3103 c 101 22
```

Check that the device is created successfully and note its file's detail:

```
ls -al /dev/sc3103
```

Now we can perform open, read, write and close with it.
Examples :

- To read from it, simply use the shell command:

```
cat /dev/sc3103
```

- To write to it*: (but don't try this for this example)

```
echo "hello" > /dev/sc3103
```

- or you could create a basic 'C' program to test it out

Testing the I/O Module

A user space C program to access the I/O module /dev/ce3103

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>

int main(int argc, char **argv)
{ char temp1[12] = "Course code"; // message to send
  static , temp2;
  int  fd,i;
  printf("opening file /dev/sc3103\n");
  fd = open("/dev/sc3103", O_RDWR);
  if (fd < 0)
     { printf("open /dev/sc3103 failed\n");
       exit (-1);
     }
  else
     printf("open /dev/sc3103 successfully\n");
```

```
 temp1 = 'E';
    i=write(fd,temp1,1);
    printf ("number of char write  = %d\n",i);
    i = read(fd,temp2,1);
    printf ("number of char read  = %i\n",i);
    if (i < 0)
       {  printf ("read error\n");
      close(fd);
      exit (-1);
        }
    else
       printf ("value read = %c\n", temp2);
    close(fd);
    exit(0);
}
```

Aside: mknod through udev

As mentioned, hardware devices are accessed through device nodes located in the `/dev` directory.

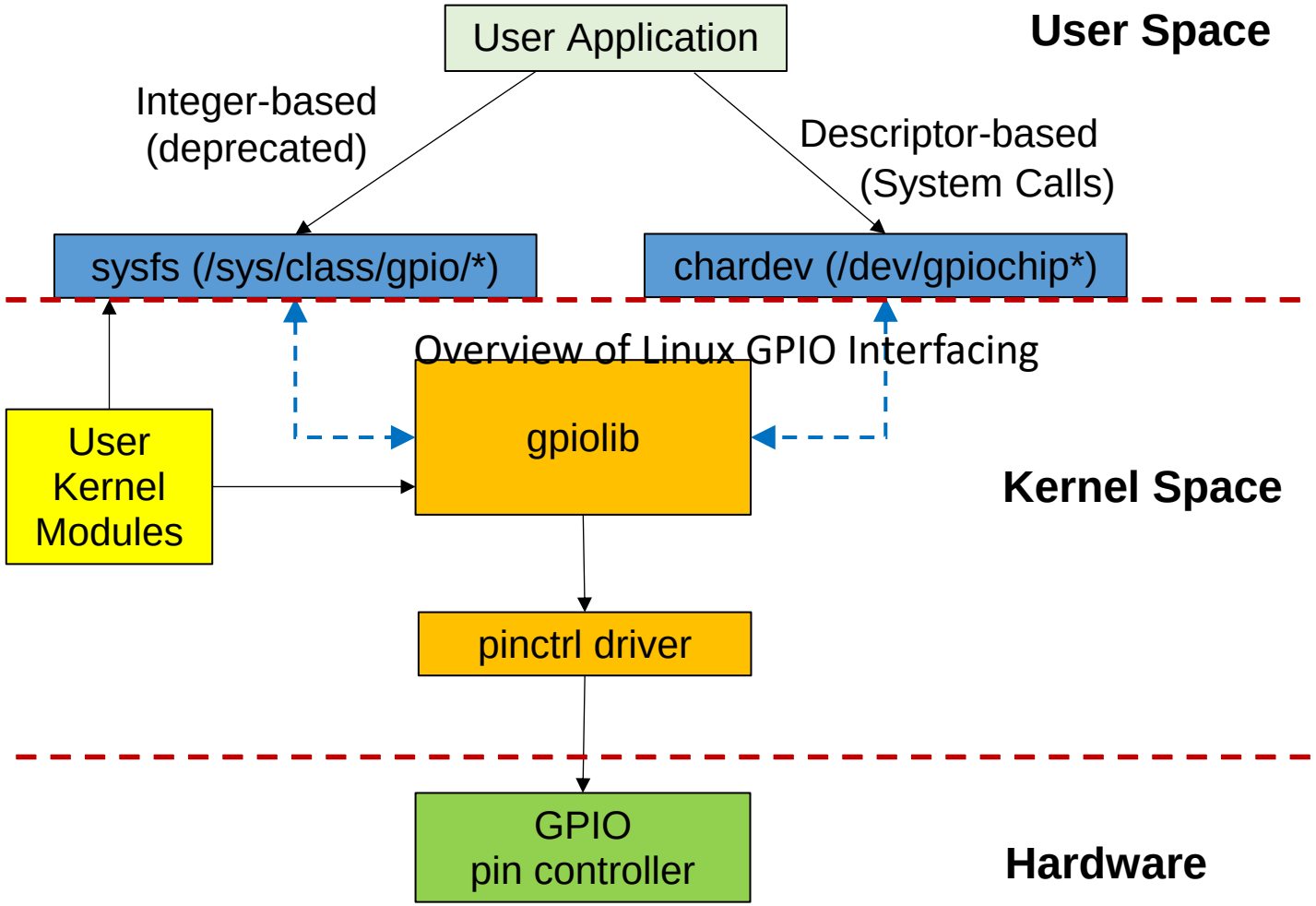
- traditionally, these special files were created at install time by the distribution, using the `mknod` command.
- so `/dev` are populated with every device that might possibly appear in the system

A better way is to use the `udev` daemon to manage these `/dev` files dynamically at runtime

- `udev` will create nodes when devices are detected and delete them when devices are removed (e.g. hotplug USB devices at runtime).
- hence the `/dev` directory contains only entries for devices which actually exist on the system at the current time (as opposed to devices which *could* exist.)

Embedded LKM
for
Interfacing to GPIO
(Integer-based Interface)

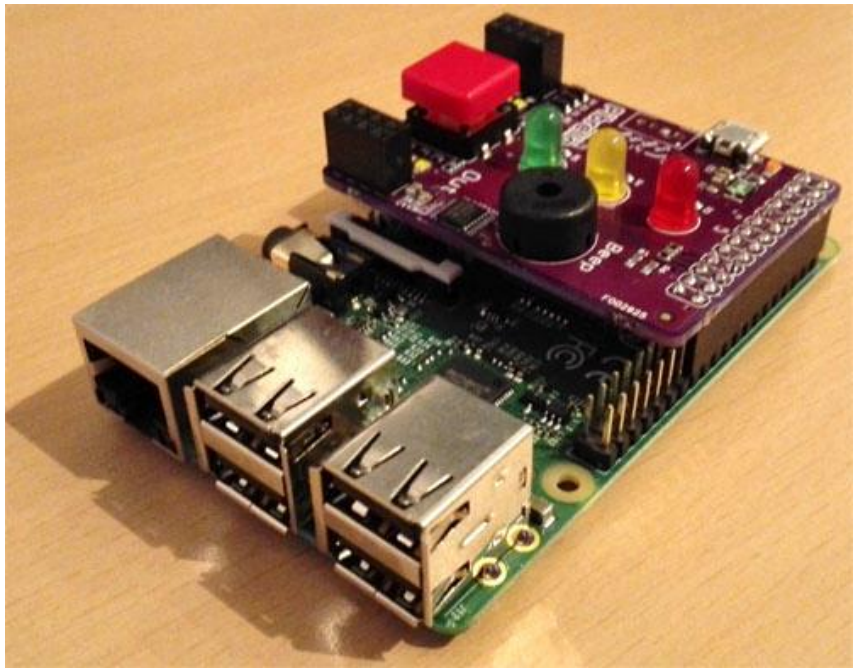
Overview of Linux GPIO Interfacing



User Space Access of GPIOs

We will learn how to develop user space application to interface with GPIO circuits using descriptor-based char device interface

- using the Pibrella board interfacing to the Raspberry Pi board as a case study



```
pi@raspberrypi:~/ce3103 $ pinout
.--------------------------------.
| oooooooooooooooooo J8     +====
| 1oooooooooooooooooo       | USB
|                           +====
|      Pi Model 3B  V1.2
|     +----+                +====
| |D| |SoC |                | USB
| |S| |    |                +====
| |I| +----+
|                    |C|    +======
|                    |S|    |    Net
| pwr        |HDMI|  |I||A| +======
`-| |--------|    |----|V|-------'

Revision           : a22082
SoC                : BCM2837
RAM                : 1024Mb
Storage            : MicroSD
USB ports          : 4 (excluding power)
Ethernet ports     : 1
Wi-fi              : True
Bluetooth          : True
Camera ports (CSI) : 1
Display ports (DSI): 1

J8:
   3V3  (1) (2)  5V
 GPIO2  (3) (4)  5V
 GPIO3  (5) (6)  GND
 GPIO4  (7) (8)  GPIO14
   GND  (9) (10) GPIO15
GPIO17 (11) (12) GPIO18
GPIO27 (13) (14) GND
GPIO22 (15) (16) GPIO23
   3V3 (17) (18) GPIO24
GPIO10 (19) (20) GND
 GPIO9 (21) (22) GPIO25
GPIO11 (23) (24) GPIO8
   GND (25) (26) GPIO7
 GPIO0 (27) (28) GPIO1
 GPIO5 (29) (30) GND
 GPIO6 (31) (32) GPIO12
GPIO13 (33) (34) GND
GPIO19 (35) (36) GPIO16
GPIO26 (37) (38) GPIO20
   GND (39) (40) GPIO21

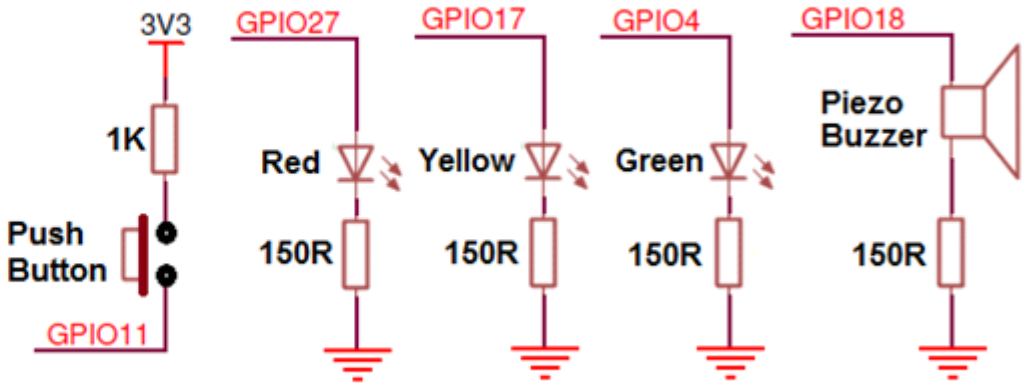
For further information, please refer to https://pinout.xyz/
```

GPIO interfaces for Pibrella

The Pibrella board consists of

- 3 LED displays interfaced through GPIO4, GPIO17 and GPIO27 of RPi board
- 1 Push button input connected through RPi's GPIO11
- 1 Piezo Buzzer driven through RPi's GPIO18





Integer-based GPIO Interface

Prior to using a GPIO from within the driver, one should claim it to the kernel
- to take the ownership of the GPIO, preventing other drivers from accessing the same GPIO.

After taking the ownership of the GPIO, one can:
- set the direction and export it to the sysfs interface
- toggle its output state if used as output
- read the state, set the debounce-interval if used as input.

For GPIO lines mapped to IRQ
- define at what edge/level the interrupt should be triggered,
- register a handler that will be run whenever the interrupt occurs.

Kernel GPIO functions

In Linux, GPIOs can be easily accessed and controlled from kernel space using the functions that are described in linux/gpio.h (of kernel source, not the user space)

```
static inline bool gpio_is_valid(int number)
static inline int  gpio_request(unsigned gpio, const char *label)
static inline int  gpio_export(unsigned gpio, bool direction_may_change)
static inline int  gpio_direction_output(unsigned gpio, int value)
static inline int  gpio_set_value(unsigned gpio)
static inline int  gpio_direction_input(unsigned gpio)
static inline int  gpio_get_value(unsigned gpio)
static inline int  gpio_set_debounce(unsigned gpio, unsigned debounce)
static inline void gpio_unexport(unsigned gpio)
static inline void gpio_free(unsigned gpio)
static inline int  gpio_to_irq(unsigned gpio)
```

ISR in Kernel Space

We can also associate an interrupt request (IRQ) with a GPIO in the kernel space using the function:

```
static inline int  gpio_to_irq(unsigned gpio)
```

To use this, we need to

- declare an ISR handler function of the type:

```
static irq_handler_t
```

Example:

```
static irq_handler_t rpi_gpio_isr(
                     unsigned int irq,
                     void *dev_id,
                     struct pt_regs *regs)
{
  :
}
```

ISR in Kernel Space

- We then register the ISR handler function with an interrupt number using the function:

```
int request_irq
```

Example:

```
request_irq(irqNumber,
            (irq_handler_t) rpi_gpio_isr,
            IRQF_TRIGGER_RISING,  // rising edge trigger
            “rpi_gpio_handler",
            NULL);
```

- To undo the IRQ request:

```
free_irq()
```

Code Snippet of LKM for RPi

Header file and declaration:

```
#include <linux/gpio.h>         // for the GPIO functions
#include <linux/interrupt.h>    // for the IRQ code
   :
static unsigned int gpioLED = 4;      //  GPIO4
static unsigned int gpioButton = 11; // GPIO11
```

Initialization function:

```
static int __init rpi_gpio_init(void)
{
  gpio_request(gpioLED, "LED-Green");       // request LED GPIO
  gpio_direction_output(gpioLED, ledOn);    // set in output mode and ON
  gpio_export(gpioLED, false);              // appears in /sys/class/gpio,
                                            // false prevents direction change

  gpio_request(gpioButton, "PushButton");   // set up gpioButton
  gpio_direction_input(gpioButton);         // set up as input
  gpio_export(gpioButton, false);           // appears in /sys/class/gpio
    :
  irqNumber = gpio_to_irq(gpioButton);      // map GPIO to IRQ number
  printk(KERN_INFO "GPIO_TEST: button mapped to IRQ: %d\n", irqNumber);
    :
```

Initialization function:

```
static int __init rpi_gpio_init(void)
{     :
   // Requests an interrupt line
   result = request_irq(irqNumber,      // interrupt number requested
              (irq_handler_t) rpi_gpio_isr,   // isr handler function
              IRQF_TRIGGER_RISING,  // on rising edge (press, not release)
              "rpi_gpio_handler",   // used in /proc/interrupts
              NULL                  // *dev_id for shared interrupt lines
              );
       :
}
```

Exit function:

```
static void __exit rpi_gpio_exit(void)
{    :
   gpio_set_value(gpioLED, 0);     // turn the LED off
   gpio_unexport(gpioLED);         // unexport the LED GPIO

   free_irq(irqNumber, NULL);      // free the IRQ number
   gpio_unexport(gpioButton);      // unexport the Button GPIO

   gpio_free(gpioLED);             // free the LED GPIO
   gpio_free(gpioButton);          // free the Button GPIO
      :
}
```

ISR handler function (triggered by gpioButton pressed)

```
static irq_handler_t rpi_gpio_isr(unsigned int irq, void *dev_id,
                                  struct pt_regs *regs)
{
   ledOn = !ledOn;                        // invert the LED state
   gpio_set_value(gpioLED, ledOn);        // set LED accordingly
   return (irq_handler_t) IRQ_HANDLED; // announce IRQ handled}       :
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

#define GPIO_PIN 4
int main()
{
    int fd;
    char buf1[256],buf2[256];

    fd = open("/sys/class/gpio/export",O_WRONLY);
    snprintf(buf1,sizeof(buf1),"%d",GPIO_PIN);
    write(fd,buf1,sizeof(buf1));
    close(fd);

    system("ls /sys/class/gpio");

    snprintf(buf2,sizeof(buf2),"/sys/class/gpio/gpio%d/direction",GPIO_PIN);
    fd=open(buf2,O_WRONLY,0644);
    write(fd,"out",3);
    close(fd);
    //system("cat /sys/class/gpio/gpio17/direction");

    int i;
    snprintf(buf2,sizeof(buf2),"/sys/class/gpio/gpio%d/value",GPIO_PIN);
    for(i=0;i<5;i++){
        printf("%s\n",buf2);
        fd=open(buf2,O_WRONLY);
        write(fd,"1",1);
        close(fd);
        sleep(1);
        fd=open(buf2,O_WRONLY);
        write(fd,"0",1);
        close(fd);
        sleep(1);
    }
    fd=open("/sys/class/gpio/unexport",O_WRONLY);
    write(fd,buf1,sizeof(buf1));
    close(fd);
    printf("Done\n");
    return 0;
}
```

Use sudo to execute the program

Summary

- Write a user app to get hardware (GPIO) access via kernel and a character device.
- Write and load a kernel module
  - Import functions from other modules
  - Introduce parameters from the users
  - Interact with user app via a character device
- Using integer-based interfacing to get hardware access